

LTE Modem System Architecture and UART Integration Guide

1. Overview

This document provides a complete technical explanation of the LTE modem subsystem used in your project. It covers all layers of the architecture:

- Hardware Layer
- Driver Layer
- State Machine Layer
- Application Layer
- Event Flow
- UART Line Parsing
- Migration from simulated modem to real hardware

This document is structured for professional engineering use and suitable for industrial IoT environments.

2. Hardware Layer (lte_hw)

The hardware layer abstracts all physical interactions with the LTE modem.

Responsibilities

- Configure UART peripheral
- Configure GPIO pins (RESET, PWRKEY)
- Provide timing functions
- Execute modem power-on sequence
- Perform hardware modem reset

Typical Functions

- `lte_hw_init()` – initializes UART and GPIO
- `lte_hw_power_on_sequence()` – toggles PWRKEY to boot modem
- `lte_hw_reset_modem()` – toggles RESET pin
- `lte_hw_ms()` – returns system millisecond tick

Power-On Sequence Example

```
void lte_hw_power_on_sequence(void)
{
    gpio_set_level(PWRKEY_PIN, 1);
    delay_ms(1000);
    gpio_set_level(PWRKEY_PIN, 0);
}
```

3. Driver Layer (lte_driver)

The driver layer handles UART communication and converts raw bytes into structured events.

Responsibilities

- Receive bytes from UART
- Detect complete modem lines (`\r\n`)
- Push events into the event queue
- Provide helper functions to check last received line
- Send AT commands

Event Types

```
typedef enum {
    LTE_EVENT_NONE,
    LTE_EVENT_RX_LINE,
    LTE_EVENT_TIMEOUT,
    LTE_EVENT_ERROR
} LteEventType_t;
```

Event Structure

```
typedef struct {
    LteEventType_t type;
    const char *line;
} LteEvent_t;
```

UART Line Detection

```
static char rx_buf[256];
static int rx_pos = 0;

void uart_rx_byte(uint8_t b)
{
    if (b == '\n') {
```

```

    rx_buf[rx_pos] = '\0';
    push_event(LTE_EVENT_RX_LINE, rx_buf);
    rx_pos = 0;
} else if (b != '\r') {
    rx_buf[rx_pos++] = b;
}
}

```

Sending AT Commands

```

void lte_driver_send_cmd(const char *cmd)
{
    uart_write_bytes(UART_NUM, cmd, strlen(cmd));
}

```

4. State Machine Layer (lte_sm)

The state machine is the core logic controlling modem behavior.

Forward Path States

INIT → POWER_ON → WAIT_BOOT → CHECK_SIM → ATTACH → PDP → OPEN_SOCKET
 → MQTT_CONNECT → MQTT_SUBSCRIBE → RUN

Recovery States

REATTACH
 REPDP
 REOPEN_SOCKET
 MQTT_RECONNECT
 MQTT_RESUBSCRIBE
 REBOOT_MODEM
 ERROR

Why Recovery States Are Needed

Industrial LTE modems frequently:

- Drop PDP context
- Drop TCP connections
- Drop MQTT sessions
- Return ERROR
- Freeze or reboot internally

Recovery states allow the modem to self-heal without restarting the entire system.

Event Handling

```
while (lte_driver_get_next_event(&ev)) {  
    lte_sm_handle_event(&ev);  
}
```

5. Application Layer (lte_app)

The application layer contains your business logic.

Responsibilities

- Periodic TCP sends
- MQTT publishes
- Alarm logic
- Sensor reading
- Telemetry

Example

```
void lte_app_loop(void)  
{  
    if (lte_sm_get_state() == LTE_SM_RUN) {  
        mqtt_publish("device/data", payload);  
    }  
}
```

6. Full Event Flow

UART RX byte

- Driver collects bytes
 - Detects \r\n
 - push_event()
 - Main loop fetches event
 - State machine processes event
 - Sends AT command
 - Modem replies
 - UART RX
 - push_event()
 - repeat
-

7. Switching from Simulation to Real Hardware

This section explains how to replace the simulated modem with a real UART-based modem.

Step 1 — Replace Fake RX with Real UART RX ISR

```
void uart_rx_isr(uint8_t byte)
{
    if (byte == '\n') {
        rx_buf[rx_pos] = '\0';
        push_event(LTE_EVENT_RX_LINE, rx_buf);
        rx_pos = 0;
    } else if (byte != '\r') {
        rx_buf[rx_pos++] = byte;
    }
}
```

Step 2 — Replace Fake TX with Real UART TX

```
void lte_driver_send_cmd(const char *cmd)
{
    uart_write_bytes(UART_NUM, cmd, strlen(cmd));
}
```

Step 3 — Implement Hardware Reset

```
void lte_hw_reset_modem(void)
{
    gpio_set_level(MODEM_RESET_PIN, 0);
    delay_ms(200);
    gpio_set_level(MODEM_RESET_PIN, 1);
}
```

Step 4 — Implement Power-On Sequence

```
void lte_hw_power_on_sequence(void)
{
    gpio_set_level(PWRKEY_PIN, 1);
    delay_ms(1000);
    gpio_set_level(PWRKEY_PIN, 0);
}
```

Step 5 — Remove Debug Injection

Delete all calls to:

```
lte_driver_debug_set_last_line()
```

Your modem is now fully real.

8. Final Summary

Your modem architecture consists of:

- Hardware Layer → UART + GPIO
- Driver Layer → line parsing + event queue
- State Machine Layer → AT command sequencing + recovery
- Application Layer → business logic

This is a professional, industrial-grade modem stack suitable for IoT and telemetry applications.

End of Document